神经网络基础

Module 3 · Topic 1 — 知识点完整整理

ARTI2001 Machine Learning · 2026 Semester 2

涵盖 **模块总览与记号表** + **Topic 1 三课**

网络结构与参数维度 · 前向传播

计算图 · 链式法则基础 · 输出神经元的损失梯度

一次梯度下降更新的完整手算

⚠ 记号表全模块通用，先看它

M3 测验：Week 13（2026-11-09 ~ 11-15）· 110 分钟 · 开卷 · 20 选择×3 + 4 大题×10

目录

M3 总览与记号表

· ★★ Module 3 记号表（跨全模块通用，考场速查）
· 模块定位
· 三个 Topic 的分工
· 模块学习目标（官方建议用它来组织复习）
· Topic 1 的前置要求与学习顺序
· Topic 1 学习目标
· ★ Tutorial 结构（10-19 当堂交，纸质）
· 可选 Python 练习（不计分）
· ⚠ TensorFlow Playground 的陷阱（与 T3 的数据泄漏直接呼应）

M3 T1 第 1 课 · 神经网络能学什么

· 1. 一个人工神经元做三件事
· 2. ★ 官方算例（要会复现）
· 3. ⚠⚠ 为什么一个神经元不够——本课最爱考的一个点
· 4. 连成网络
· 5. ★★ 为什么层间必须有非线性——带数字的塌缩论证
· 6. 网络学习时改变的是什么
· 7. 其它要知道的术语
· ⚠ TensorFlow Playground 的一个坑（页面专门提醒）

M3 T1 第 2 课 · 反向传播基础：计算图与梯度

· 1. 从预测到学习目标
· 2. ★★ 反向传播到底是什么（最容易答错的概念题）
· 3. 计算图
· 4. 导数 = 敏感度
· 5. ★★ 链式法则：沿一条路径**相乘**
· 6. ★★ 一个值影响多条路径时：把贡献**相加**
· 7. 把反向计算接到损失上
· 8. 四道自测题的要点

M3 T1 第 3 课 · 单个神经元怎么学（含完整的一次更新算例）

· 0. 这一课的主线
· 1. 二元交叉熵 BCE
· 2. ★★★ 学习信号 δ = p − y（全课最该记的公式）
· 3. ★★ 一次梯度下降更新
· 4. 配套活动：Computational graph by hand
· 5. 两道总结自测题
· ★ 速查卡（考场上最常用的一串）

M3 总览与记号表

源页面：Module 3 overview + Module 3 Topic 1 overview | 整理日期：2026-10-13

★★ Module 3 记号表（跨全模块通用，考场速查）

下标 1 / 2 加在**矩阵**名上表示**第几层**；加在小写符号上表示**第几个单元或输入**。

符号	含义
x（元素 x_i）	一个观测的输入列向量
W_1, b_1	隐藏层权重矩阵与偏置列；**w_ji = 从输入 i 到隐藏单元 j 的权重**
z（元素 z_j）	隐藏层分数 / 预激活：**$z = W_1x + b_1$**
h（元素 h_j）	隐藏层输出：**$h = g(z)$**，g 是隐藏激活（如 ReLU）
m	隐藏单元个数
$W_2 = [v_1 \dots v_m]$, c	输出层权重与输出偏置
s	输出分数 / **logit**：**$s = W_2h + c$**
p	类别 1 的预测概率：**$p = \sigma(s)$**
y, ŷ	观测目标；ŷ 是数值预测（如回归里）
L, δ	损失；**$\delta = \partial L/\partial s$ 是输出学习信号**（sigmoid 输出 + BCE 时 **$\delta = p - y$**）
η	梯度下降更新里的学习率

⚠ 两个容易栽的记号陷阱

1. **单神经元的课**写的是 `z = wᵀx + b`、`a = g(z)`；**多层的课**用上面那套 z/h/s/p。别混。
2. **代码里**名为 `W1`、`W2` 的数组存的是 **W_1^T 和 W_2^T**（转置），因为数据数组里**每一行是一个观测。**
3. 损失记号跨 Topic 会变：**T1 用 L 表示单个观测的损失；T2 用 ℓ 表示单观测、用 L 表示多观测的平均。**

模块定位

- **Module 1 / 2 按「训练信息」分类**（监督 / 无监督）；**Module 3 讲的是一个模型家族，不是第三种学习范式。**
- 神经网络在监督、无监督、以及自监督（目标由数据自己构造）场景下都能用。
- 本课核心聚焦：**用于监督预测的小型前馈网络**；更高级的架构只作背景了解。
- **前馈（feed-forward）** = 信息从输入经隐藏层流向输出，**没有回环。**
- **深度（deep）** 指这种层层复合，**不是**指像人一样思考。

三个 Topic 的分工

Topic	内容
T1 理解网络与一次学习步	神经元、前向预测、计算图与反向传播基础，**以单神经元的一次更新收尾**
T2 训练多层网络	激活/损失的选择、隐藏层梯度、批量、实现与自动微分
T3 评估与改进泛化	数据划分、学习率与训练证据、停止决策、分类指标与 ROC/AUC

模块学习目标（官方建议用它来组织复习）

1. 描述神经网络的基本架构与组成部分，**包括激活函数和损失函数。**
2. 为给定任务**实现并训练**一个简单神经网络。
3. 应用恰当的**模型评估**技术，解释**泛化、过拟合与欠拟合。**

> 「实现」这条怎么被支撑：T1 做出单神经元的一次学习步 → T2 把前向/反向扩展到隐藏层和批量 → **T2 的 practical 提供有引导的实现与训练流程**，T2 overview 解释方程与代码的对应并给出独立的实现校验 → T3 提供评估原则。**要按这个顺序走。**

Topic 1 的前置要求与学习顺序

开始前应该已经会：

- 计算加权和、知道偏置的作用
- 检查基本矩阵乘法的维度
- 回忆 M1 的 sigmoid，并**区分概率和阈值判出的类别标签**
- 区分「观测到的输入与目标」和「预测与可训练参数」

> 不需要事先懂反向传播，第 2 课会introduce所需的导数与链式法则。

学习顺序：

1. What can a network learn? → 神经元、层、非线性表示
2. Neural network playgrounds → 分类与回归两种任务里探索预测
3. Backpropagation foundations → 损失、敏感度、链式法则 → 然后做 **Computational graph by hand**
4. How one neuron learns → 算损失梯度、做一次更新、复查结果

预计学习时间：约 **3.5–4 小时**（不含排定的 tutorial 和 practical）。

> ⚠ **不要求在 Topic 2 之前会算隐藏层梯度。**

Topic 1 学习目标

1. 描述一个人工神经元和一个前馈网络
2. 把相互依赖的计算表示成**计算图**

3. 计算小网络的一次**前向传播**
4. 对单个神经元应用**链式法则**求局部与端到端导数
5. 解释**前向传播、反向传播、梯度下降三者各自不同的角色**，并完成一次单神经元更新

★ Tutorial 结构（10-19 当堂交，纸质）

> 这是**限时堂内测评**：tutor 当场发纸质 worksheet，课上做完、离开前交给 tutor，**不要线上提交。**

题	内容
Q1	算网络**前向传播**；并问**去掉隐藏层 ReLU 会怎样**
Q2	用**链式法则**推导一个输出梯度，然后算出**全部**输出梯度并解读
Q3	**只更新输出层**，复查新的预测和损失。⚠ 这次更新中**隐藏层激活保持固定**
扩展	一道可选、不计分的题，换一个目标标签

> **⚠ 隐藏层梯度在 Topic 2 的 worksheet 里单独练。**
> 解答在**下一周的周三**发布。

> ★ 排课说明：原本的 practical 时段被 Module 2 Assessment 占用，**没有单独排 M3 T1 practical。**

可选 Python 练习（不计分）

一个 notebook 贯穿整个模块，文件名叫 M3 T1 只是标明出处，不是要求在 T1 做完。

题	什么时候做
Q1 前向传播	**现在**
Q2、Q4、Q5 隐藏层梯度、失活的 ReLU、自动微分	**Topic 2 之后**
Q3 学习率比较	**Topic 3 的训练证据那节之后**

> 总计约 90 分钟 + 30 分钟检查，跨这几个阶段。**同一个 notebook 存着接着用。**

⚠ TensorFlow Playground 的陷阱（与 T3 的数据泄漏直接呼应）

Playground 把它留出的那条曲线标成 **"test loss"**。

> **如果你反复用那条曲线来挑设置，它起的作用就是验证集，而不是一个没被碰过的最终测试集。**

它只是可视化探索工具，**输出与损失的约定不一定和本课的 sigmoid/BCE 算例一致。**

M3 T1 第 1 课 · 神经网络能学什么

源页面：Neural network introduction: What can a network learn? | https://learn.adelaide.edu.au/courses/30362/pages/neural-network-introduction-what-can-a-network-learn | 整理日期：2026-10-13 | 对应考纲：Topic 1「网络结构与参数维度、前向传播」

1. 一个人工神经元做三件事

加权 → 加偏置 → 过激活函数。

> $z = w_1x_1 + w_2x_2 + b$　　（pre-activation，预激活 / 分数）
> $a = g(z)$　　（g 是激活函数，a 是输出）

量	含义	⚠ 是不是参数
x_1, x_2	观测到的输入特征，由样本给定	否，是数据
w_1, w_2	权重，每个输入的可调贡献	**是**，训练时会变
b	偏置，与当前输入无关的可调偏移	**是**
z	预激活：加权和加偏置	否，算出来的
a	激活输出	**否**——⚠ 它随输入而变，**不是存下来的参数**

- **权重的方向性**：正权重 ⇒ 该输入变大时分数变大；负权重 ⇒ 反之。
- **偏置**：不改变输入，只平移分数。

⚠ 考点易混：`a` 是「算出来的中间值」，不是参数。问「优化器调整哪些量」时，答案只有**权重和偏置**。

2. ★ 官方算例（要会复现）

给定 **x = [1, 2]**，目标 **y = 1**，初始 **w_1 = 0.2, w_2 = 0.4, b = 0.1**：

> 加权：　$0.2 \times 1 = 0.2$　　$0.4 \times 2 = 0.8$
> 求和：　$z = 0.2 + 0.8 + 0.1 = 1.1$
> 激活：　$p = \sigma(1.1) = 1/(1+e^{(-1.1)}) \approx 0.750260$
> 判类：　规则「$p \geq 0.5$ 判类 1」 ⇒ 预测类别 1

★ 概率(≈0.75) 和类别标签(1) 是两个不同的量

，考试爱考这个区分。

概率也写作 P(y=1 | x)，读作「给定输入 x 时类别为 1 的概率」。它是模型的**估计**，不是正确性的保证。

这整个过程叫前向传播（forward pass）：信息用**当前参数**从输入流到输出。

⚠ **没有任何参数改变，所以还没有发生学习。** 目标 y 在这一步**没有被用到**——它是训练时用来评判这个预测的。

自测题（页面给的）

1. 只把 b 从 0.1 提到 0.6，其余不变 ⇒ z = 1.6，概率**上升**。
2. 某模型给出 p = 0.99 —— **光凭这个数不能断定**它的参数是从数据学来的，**也不能断定预测是对的**。高置信度 ≠ 学过 ≠ 正确。

3. ⚠⚠ 为什么一个神经元不够——本课最爱考的一个点

决策边界必然是直线

预测规则是「p ≥ 0.5 判类 1」。而 **sigmoid 恰好在 z = 0 时等于 0.5**，所以决策边界是：

> $w_1x_1 + w_2x_2 + b = 0$

两个输入时，这个方程描述的是**一条直线**（只要权重不全为 0）。改权重和偏置能**平移或旋转**这条线，**但不能把它弄弯，也不能变出好几条线。**

（若两个权重都为 0，预测对任何输入都是常数，同样解决不了。）

★★ 「sigmoid 是非线性的，边界怎么还是直的？」

这是页面专门拎出来讲的一处，必考：

问的是什么	答案
概率输出是不是输入的非线性函数?	**是。**sigmoid 让 p 对输入非线性。
决策边界（预测类别发生改变的地方）是什么形状?	**仍是直线。**因为在阈值 0.5 处，类别改变恰好发生在 **z = 0** 这个位置。

> 原理：**sigmoid 保序**——正分数给出 >0.5 的概率，负分数给出 <0.5 的概率。所以「哪里越过 0.5」完全由「哪里 z 越过 0」决定。

什么样的模式一个神经元搞不定

- **能搞定**：蓝点全在左、橙点全在右 ⇒ 一条竖线分开。

例：$w_1 = -1$, $w_2 = 0$, $b = 0$ ⇒ $z = -x_1$，蓝点分数为正、橙点为负。

- **搞不定**：两类各占**对角线上的两个角**（异或型）。任何一条直线都会把类别混在一起。

> ⚠ **关键推论**
>
> 如果一个 sigmoid 神经元在这种模式上一直出错，**训练更久也不可能全分对。**
>
> 这是**模型表达能力的限制**，不是训练不足。「再train久一点」是错误选项。

4. 连成网络

- **层（layer）**：计算中处于同一阶段的一组单元。
- **输入层**：提供特征值。
- **隐藏层**：在输入与最终预测之间计算中间值。「隐藏」指这些是**内部值**，既不是观测到的输入特征、也不是要求的输出。

⚠ 权重下标的读法（容易记反）

> w_{ji} = 从输入 i 到隐藏神经元 j 的权重
> 第一个下标 = 接收的隐藏神经元
> 第二个下标 = 输入

所以 **w_{11} 和 w_{12} 喂给 h_1，w_{21} 和 w_{22} 喂给 h_2。**输出权重 v_1、v_2 分别乘 h_1、h_2。

- 每条有标注的连线都是一个**独立的可调参数。**
- 每个计算型神经元都有**自己的**权重和偏置。
- ⚠ 图上交叉的箭头**不构成额外的神经元。**
- 神经元里标的字母是**它的输出名**，不是它的权重。

隐藏激活是什么

隐藏激活是**用学到的权重算出来的中间特征。**不同的权重让不同神经元响应不同的输入组合。

> ⚠ 这些特征**不一定对应人能读懂的概念**（比如"某个传感器坏了"）。
>
> ⚠ 激活是**针对每个样本重新算的**，它**本身不是存下来的权重。**

本课用的激活函数搭配

- **隐藏层：ReLU**，g(z) = max(0, z) —— 「负分数输出 0，否则原样输出」。ReLU(−2) = 0，ReLU(2) = 2。
- **输出层（二分类）：sigmoid。**

5. ★★ 为什么层间必须有非线性——带数字的塌缩论证

光堆层数没用。 如果每层只做"乘权重加偏置"，整串可以改写成**一个**加权和加偏置。

不加激活函数（会塌缩）

设一个输入 x、一个隐藏值 h、最终分数 s：

> 第一层：$h = 2x + 1$
> 第二层：$s = 3h + 4$
> 代入：$s = 3(2x + 1) + 4 = 6x + 3 + 4 = 6x + 7$

> ⇒ **一个权重为 6、偏置为 7 的单层，对任何输入都给出完全相同的输出。**
>
> 两层算下来**还是一条直线。**

术语：「加权和 + 偏置」叫**仿射函数（affine function）**。

「层塌缩成一个仿射函数」就是说——整串可以被那一个等价计算替换掉。

插入 ReLU（不再塌缩）

权重偏置完全不变，只让隐藏神经元把负分数置零：

$$h = \max(0, 2x + 1) \qquad s = 3h + 4$$

条件	h	s
$x \le -0.5$（此时 $2x+1 \le 0$）	0	$s = 3\times0 + 4 = \mathbf{4}$
$x > -0.5$（此时 $2x+1 > 0$）	$2x + 1$	$s = \mathbf{6x + 7}$

两段在 **$x = -0.5$** 处相接，该点 **$s = 4$**。左边是平的，右边上升——**出现了一个折点。**

> ★ **所以没有任何单个仿射计算能对所有输入复现这个函数。**
>
> 每一段都是直线，但整个函数不是一条直线。

自测：$x = -1$ 时，有 ReLU 得 $s = 4$，无 ReLU 得 $s = 1$；$x = 1$ 时两者都是 13。

（$x=1$ 落在 ReLU 不起作用的区域，所以一致；$x=-1$ 落在被截断的区域，所以不同。）

⚠ 「只在最后放个 sigmoid 不行吗？」

不行。 没有非线性的隐藏激活，**送进最后那个 sigmoid 之前的分数仍然是原始输入的仿射函数。**sigmoid 让概率输出变成非线性，但它的 0.5 阈值仍然对应分数为零——两个输入时仍是**一条直的边界。**

6. 网络学习时改变的是什么

监督训练反复做这个循环：

步骤	做什么
Predict	用当前权重和偏置做一次前向传播
Compare	用已知目标和**损失函数**（预测误差的数值度量）评判这个预测
Adjust	用「参数如何影响损失」的信息更新权重和偏置
Repeat	重新计算预测，在训练样本上继续

⚠ 考点：一次更新中到底什么变了

对固定的一个训练样本 x = [1,2]、y = 1：

问题	答案
优化器调整哪些**存下来的**量?	**只有权重和偏置**
更新之后，同样的输入再过一遍，激活和预测会变吗?	**会变**——因为参数变了
我们会改输入值或目标标签来降低误差吗?	**绝对不会**

★ 网络学到的"知识"是**分布在全部权重和偏置里**的；

单个神经元一般不存储一条完整的事实。

⚠ 表达能力更强**不保证**训练成功，也**不保证**在新数据上预测得好——必须在训练用过的样本之外评估。

7. 其它要知道的术语

- **前馈网络（feed-forward）**：数值从输入经隐藏层流到输出，**没有回环。**
- **深度（deep）**：指这种**层层复合**，**不是**指像人一样思考。
- **优化器（optimiser）**：更新参数的那条规则。

⚠ TensorFlow Playground 的一个坑（页面专门提醒）

Playground 把它留出的那条曲线标成 **"test loss"**。

如果你反复用那条曲线来选设置，它起的就是验证集的作用，而不是一个没被碰过的最终测试集。

这一点和 Topic 3 的「数据泄漏」直接呼应。另外它的输出与损失约定不一定和本课的 sigmoid/BCE 算例一致。

M3 T1 第 2 课 • 反向传播基础：计算图与梯度

源页面：Backpropagation foundations: computational graphs and gradients ｜ https://learn.adelaide.edu.au/courses/30362/pages/backpropagation-foundations-computational-graphs-and-gradients ｜ 整理日期：2026-10-13 ｜ 对应考纲：Topic 1「计算图、链式法则基础」

1. 从预测到学习目标

前向传播用**当前**权重和偏置算出一个预测。它**既不告诉你这个预测有没有用，也不改变参数。**

损失函数拿 p 和已知目标 y 比较，返回一个数（单个样本记作 L）。**损失越小 = 按所选损失函数衡量匹配得越好。**

★ 页面给的区分例子：当 **y = 1** 时，本课用的二元交叉熵给 **p = 0.9 的损失比 p = 0.6 更小。**

⚠ 注意：**两个预测在 0.5 阈值下都判类 1**，但损失能区分它们的概率估计。

若 y = 0，偏好反过来。

⇒ **目标和损失共同定义了「改进」是什么意思。**

⚠ 但这只是针对**一个**带标签样本；光有自信的预测**不能**说明在新数据上表现好。

训练的四步

步骤	内容
Predict	用当前参数跑一次前向
Measure loss	把预测和已知目标比较
Compute gradients	求「每个权重和偏置的微小变化会怎样影响损失」
Update	用这些梯度调整参数，然后用新的前向重复

⚠ 自测题：把 p 调大总是改进吗?

不是——取决于 y。y=0 时把 p 调大反而让损失变大。

2. ★★ 反向传播到底是什么（最容易答错的概念题）

一个网络可能有很多权重偏置。隐藏层的权重是**通过后面的神经元间接**影响最终预测的。我们需要一个系统的办法去追踪这些影响，而不是靠猜、也不是把每个参数单独试一遍。

- **导数**衡量「输出对输入的微小变化有多敏感」。
- 损失依赖很多参数时，它的**梯度**把每个参数各自的那一份敏感度收集到一起。

- **反向传播**是高效算出这些梯度的方法：前向跑完之后，**从损失往回**穿过产生它的那些运算，用**链式法则**把相继运算的敏感度组合起来。它**复用中间结果**，所以共享的部分不用重复算。

⚠⚠ 两件不同的工作——必考

	干什么
反向传播	**计算**梯度
梯度下降（更新方法）	**使用**这些梯度去改变参数，朝损失下降的方向迈一个合适大小的步子

> ⚠ **跑一次反向传播本身不改变任何权重。**
> ⚠ 它**也不是**把前向的算术倒着跑一遍去重建输入。

3. 计算图

计算图是「运算及其依赖关系」的示意图，不是数据点的散点图。 运算节点做小计算，箭头表示它们需要哪些值。这让「从一个权重到预测」的路线变得可见。

沿用同一个神经元：$x_1=1$, $x_2=2$, $w_1=0.2$, $w_2=0.4$, $b=0.1$

> $u = w_1x_1$，$v = w_2x_2$，$z = u + v + b$，$p = \sigma(z)$

阶段	**运算**	**值**
第一支	u = 0.2 × 1	**0.2**
第二支	v = 0.4 × 2	**0.8**
合并	z = u + v + 0.1	**1.1**
变换	p = σ(z)	**0.750260**

> ⚠ **u 和 v 是临时的中间计算值，不是输出层权重。**
> （别和 v_1、v_2 混。）

记号约定

- **Δ**（大写 delta）表示"一个变化量"：Δu 是输入的微小变化，Δf 是由此产生的输出变化。
- **′**（撇）表示导数，如 f′(u)。
- **∂** 用于**偏导数**——函数有好几个独立输入时，**其它独立输入保持不变。**
- **≈** 表示约等于。
- **d**（如 dF/dq）表示 F 作为单个变量的函数时的普通导数。

4. 导数 = 敏感度

导数回答："在当前值附近，这个输入每变化一个单位，输出变化多少？"

- **正导数** ⇒ 小幅增加会**抬高**输出
- **负导数** ⇒ **降低**输出
- **大小**描述这个效应在局部的强弱
- ⚠ **它不规定我们应该把参数更新多远。**

官方的标量例子（说明「局部近似」）

f(u) = u²，在 u = 3 处斜率 f′(3) = 2×3 = **6**。

> u: 3 → 3.01　　f: 9 → 9.0601　　实际变化 0.0601
> 导数估计的变化：　$6 \times 0.01 = 0.06$

> ★ 差那一点点，是因为**斜率本身沿着这条弯曲的函数在变**。这是**局部近似。**

> ⚠ 对照：前面那个**仿射**的分数计算里，变化是**精确的**（因为斜率不变）。

★ 局部导数对照表（要背）

运算	局部导数	含义
r = a + b	∂r/∂a = 1；∂r/∂b = 1	**加法原样传递**变化
r = a·b	∂r/∂a = b；∂r/∂b = a	**另一个因子**决定缩放倍数
r = a²	∂r/∂a = 2a	敏感度取决于当前值
p = σ(z)	**∂p/∂z = p(1−p)**	sigmoid 递增，敏感度取决于**它的输出**

> ⚠ 表里的 a、b 是通用的运算数，**不是**别处那个激活 a 和偏置 b。

5. ★★ 链式法则：沿一条路径相乘

一个权重的影响必须穿过它和最终概率之间的每一个运算。

路径：$w_2 \to v \to z \to p$

$$\partial p/\partial w_2 = (\partial p/\partial z) \cdot (\partial z/\partial v) \cdot (\partial v/\partial w_2)$$
$$= p(1-p) \cdot \quad 1 \quad \cdot \quad x_2$$

代入数字：

$$p(1-p) = 0.750260 \times 0.249740 \approx 0.187370$$
$$\partial p/\partial w_2 = 0.187370 \times 1 \times 2 \quad \approx 0.374740$$

⇒ w_2 增加 0.01，p 大约增加 **0.003747**。

★ 官方做的「近似校验」（很能说明问题）

> 新分数 $z = 1.12 \Rightarrow p \approx 0.753989$
> 实际增量 ≈ 0.003729　　导数预测 0.003747

> 二者**接近但不相同**，因为 **sigmoid 是非线性的，斜率在这一步里略有变化。**

> ⚠ 正的导数告诉我们"在这里增大 w_2 会抬高 p"，但**还没告诉我们这样做是否可取**——因为我们还没把 p 和目标比较。

反向读图（从右往左）

> 起点 $\partial p/\partial p = 1$　　（p 自己变一个单位，自己就变一个单位）
> 过 sigmoid　$\times 0.187370$
> 过 加法　$\times 1$
> 过 乘法 $v=w_2x_2$　$\times x_2 = 2$
> ———————————————
> 累积结果　$\partial p/\partial w_2 \approx 0.374740$

w_1 和偏置那两支同理可追。

局部 vs 端到端

- **$\partial p/\partial z$** 只描述 sigmoid 那一步 —— **局部**
- **$\partial p/\partial w_2$** 包含从权重到概率的**整条路径** —— **端到端**

6. ★★ 一个值影响多条路径时：把贡献相加

相乘只在一条路径内部用。 如果一个量通过**好几条**路径影响结果，还必须把这些路径的效应**加起来**。

官方的小例子：共享值 q 喂给两支，再合并

> $q \longrightarrow r = 2q \longrightarrow$
> $\qquad\qquad \longmapsto F = r + t$
> $q \longrightarrow t = 3q \longrightarrow$
>
> $dF/dq = (\partial F/\partial r)(dr/dq) + (\partial F/\partial t)(dt/dq)$
> $\quad = \ 1 \times 2 \ + \ 1 \times 3 \ = 5$

校验：q = 1 时 F = 5；q = 1.01 时 F = 5.05。✓

> ⚠ **只沿 r = 2q 那一支算，会得到 2，漏掉另一支的贡献。**

★ 在网络里，**一个隐藏激活可能喂给后面多个单元**，所以**它的梯度要收集所有这些路径的贡献。**

7. 把反向计算接到损失上

前面算的是 $\partial p/\partial w_2$（权重怎样影响**概率**）。训练需要的是 **$\partial L/\partial w_2$**（权重怎样影响**损失**）。

$\partial L/\partial w_2$ 的意思是：**其它参数和给定的观测保持不变**时，损失对 w_2 的局部变化率。

把前向依赖链延长一节：**$w_2 \to v \to z \to p \to L$**（目标 y 也是损失计算的一个输入）。

反向就多出一个局部导数：

$$\partial L/\partial w_2 = (\partial L/\partial p) \times (\partial p/\partial w_2)$$

★ 符号分析（考点）

因子	在本例中	理由
$\partial L/\partial p$	**负**	y = 1 且用 BCE 时，**增大 p 会降低损失**
$\partial p/\partial w_2$	**正**	刚算出来是 +0.374740
乘积 $\partial L/\partial w_2$	**负**	⇒ **对这个观测，略微增大 w_2 会降低损失**

⚠ 这是**局部方向**，**不保证**大步更新也成立。

反向传播的完整工作方式

1. 从选定的最终损失开始，**$\partial L/\partial L = 1$**
2. 往回累积敏感度
3. 每经过一个运算就**乘**上局部导数
4. 路径汇合处就**加**上各自的贡献
5. **用的是同一次前向传播留下的值**

★ 只有在参数梯度全部算完之后，梯度下降才去改变权重和偏置。

8. 四道自测题的要点

问题	要点
前向值和导数有什么区别?	前向值是**算出来的量**；导数是**敏感度**，描述改变某个输入会怎样影响输出
计算图怎样为训练做准备?	它让**从每个参数到损失的路线**变得可见，从而能系统地反向追踪敏感度并复用中间结果
反向传播和梯度下降各贡献了什么?	**反向传播算梯度；梯度下降用梯度改参数。**两者是不同的工作
只有 $\partial p/\partial w_2$ 够不够决定一次更新?	**不够。**它没和目标比较过，缺 $\partial L/\partial p$；而且它也不规定步长

M3 T1 第 3 课 • 单个神经元怎么学（含完整的一次更新算例）

源页面：How one neuron learns + M3 T1 Computational graph by hand ｜ 整理日期：2026-10-13 ｜ 对应考纲：Topic 1「输出神经元的损失梯度、一次梯度下降更新」 ｜ ★★ **这一课是 10-19 tutorial Q2/Q3 和考试大题的直接来源**，算例要会从头到尾复现。

0. 这一课的主线

> 预测 → 损失 → 梯度 → 更新 → 复查

沿用同一个神经元：**x = [1, 2]，y = 1，w = [0.2, 0.4]，b = 0.1**

⇒ 分数 z = 1.1，类别 1 概率 p = 0.750260

> 阈值判类已经**对了**（0.750 ≥ 0.5 ⇒ 判 1），但模型仍把一部分概率分给了错误的类。

> **★ 损失提供了一个连续的学习信号，即使类别标签不再变化。**

> ★ 这个设定其实就是**把逻辑回归看成一个可训练的神经元。**

1. 二元交叉熵 BCE

> $L = -[\, y \cdot \log p + (1-y) \cdot \log(1-p) \,]$　　（自然对数）

y 的取值	化简成	含义
y = 1	**L = −log(p)**	给类别 1 的概率越大，损失越小
y = 0	**L = −log(1−p)**	反过来

怎么读这个式子：y 非 0 即 1，所以**两项里只会剩一项。**(0,1) 区间内概率的自然对数是负的，前面那个负号让损失变成正的。**给观测到的那一类分配低概率 ⇒ 惩罚更大。**

⚠⚠ 两个必考的坑

1. **损失里用的是概率 p，不是阈值判出来的类别标签。**

> **绝对不要**
>
> 先把 p 四舍五入成类别标签再取对数。

1. y = 1 时的数值感：

p	L = −log(p)
0.5	≈ **0.6931**
0.750260	≈ **0.287335** ← 本例
0.9	≈ **0.1054**

★ p = 0.75 和 p = 0.90 **在 0.5 阈值下都判类 1**，但 BCE 给 0.90 更小的损失

⇒ **这就是「对/错」之外多出来的信号。**

⚠ 但这只是对一个**带标签样本**的目标函数，**并不是说在没见过的或有噪声的数据上，把置信度推向 1 总是合适的。**

⚠ 记号约定（跨 Topic 容易混）

记号	本课（T1）	Topic 2
单个观测的损失	**L**	ℓ（小写 ell）
多个观测的平均损失	—（本课只有一个观测，不需要平均）	**L**

- **logit** = 过 sigmoid 之前的原始分数，这里就是 **z**。
- **δ**（小写 delta）是一个**有名字的损失梯度**；⚠ 和表示"有限变化量"的 **Δ** 不是一回事。
- 带下标 i 的导数里，**i 选的是某个输入特征和它的权重**（这里 i = 1 或 2）。

2. ★★★ 学习信号 δ = p − y（全课最该记的公式）

损失梯度衡量损失对某个参数或分数的微小变化如何响应。**负导数**意味着该量**略微增大会在局部降低损失。**

⚠ 注意区别：第 2 课求的是**概率**的导数，这一课求的是**损失**的导数。

推导（页面说明这是为了解释、不是让你背）

y = 1 时追两个运算：

$$L = -\log(p) \quad \Rightarrow \partial L/\partial p = -1/p \quad \approx -1.332871$$
$$p = \sigma(z) \quad \Rightarrow \partial p/\partial z = p(1-p) \quad \approx 0.187370$$

链式相乘：

$$\partial L/\partial z = (-1/p) \times p(1-p) = -(1-p) = p - 1 \approx -0.249740$$

结论：对 sigmoid 输出 + BCE，链式法则把「对分数的导数」化简成预测减目标

$$\delta = \partial L/\partial z = p - y \approx 0.750260 - 1 = -0.249740$$

★ δ 是**损失的敏感度，不是一个新的预测值。**

再往回一步，得到参数梯度

因为 $z = w_1x_1 + w_2x_2 + b$，所以 **$\partial z/\partial w_i = x_i$、$\partial z/\partial b = 1$**：

$$\partial L/\partial w_i = \delta \cdot x_i \qquad \partial L/\partial b = \delta$$

参数	损失梯度	解读
w_1	$-0.249740 \times 1 =$ **-0.249740**	略微增大会降低损失
w_2	$-0.249740 \times 2 =$ **-0.499480**	局部效应是 w_1 的**两倍**
b	**-0.249740**	略微增大会抬高分数、降低损失

⚠⚠ **两个权重的梯度不同，只是因为两个输入的数值不同（$x_1=1$, $x_2=2$）。**

这里梯度绝对值更大，不构成「特征更重要」的一般性排序。 —— 这是页面专门警告的一个误读。

3. ★★ 一次梯度下降更新

$$\theta_new = \theta - \eta \cdot (\partial L/\partial \theta)$$

θ 代表任一个可训练参数（w_1、w_2 或 b），η 是**学习率**（步长）。

★★ 所有梯度都必须用更新前的原始参数值算。

梯度指向损失**局部上升**的方向，**减去**它就是朝相反方向走。

取 η = 0.1 的完整计算

$$w_2,new = 0.4 - 0.1 \times (-0.499480) = 0.4 + 0.049948 = 0.449948$$
$$w_1,new = 0.2 - 0.1 \times (-0.249740) = 0.2 + 0.024974 = 0.224974$$
$$b,new = 0.1 - 0.1 \times (-0.249740) = 0.1 + 0.024974 = 0.124974$$

⚠ **减去一个负数 = 增大。**

三个梯度都是负的，所以三个参数都变大。

⚠⚠ **先把三个梯度全部算完，再从同一个状态一起更新这三个参数**（不能算一个更一个）。

更新后复查（要会算这张表）

> 新分数 $z = 0.224974 \times 1 + 0.449948 \times 2 + 0.124974 \approx 1.249844$
> 新概率 $p = \sigma(1.249844) \approx 0.777273$
> 新损失 $L = -\log(0.777273) \approx 0.251964$

量	更新前	一次更新后
分数 z	1.100000	**1.249844**
类别 1 概率 p	0.750260	**0.777273**
BCE 损失	0.287335	**0.251964**

> ★ **概率朝目标移动，损失下降。类别标签仍然是 1，但损失改善了。**

> ⚠⚠ 这**只证明在这一个训练样本上有改进，不证明模型能泛化到新观测。**
>
> ⚠ 要再走一步，必须**重新跑前向、重新算梯度。**

> ★ 展示的数值是四舍五入过的，**计算过程中要保留完整精度。**

两道自测题

问题	答案
如果 w_1 更新时用了**加号**会怎样?	得到 0.2 + (−0.024974) = 0.175026，**局部上会让损失变大**（走错了方向）
这次更新应该同时改变 x 或 y 吗?	**绝对不。**只有权重和偏置被更新；输入和目标是给定的数据

4. 配套活动：Computational graph by hand

用的是同一个模型（x_1=1, x_2=2, w_1=0.2, w_2=0.4, b=0.1）。

⚠ **这个活动的最终量是概率 p，还没接损失**——是在接损失之前先练敏感度。

> $u = w_1 x_1$，$v = w_2 x_2$，$z = u + v + b$，$p = \sigma(z)$

三个任务：

1. **画图并计算**：画出两条乘法支汇合到 z、再喂给 sigmoid；标出观测输入、参数、中间值；算 u, v, z, p。
2. **先预测再追一条路径**：不先算，先判断增大 w_2 会抬高还是降低 p；然后算 $\partial p/\partial w_2$，并估计 w_2 增加 0.01 的效果。
3. **解释学习还需要什么**：知道「怎样增大 p」**并不能**告诉我们这个改变是否改进了预测——还需要**目标 y 和损失。**

5. 两道总结自测题

Q：反向传播和梯度下降各贡献了学习的哪一部分？

> 反向传播**计算**梯度（用链式法则从损失往回累积敏感度）；梯度下降**使用**这些梯度、按学习率决定的步长**改变参数**。反向传播本身不改变任何权重。

Q：这次成功的更新是否说明一个神经元能学会任何模式？

> **不能。**
>
> 这只说明在这一个样本上损失下降了。第 1 课已经证明：
>
> **单个 sigmoid 神经元的决策边界必然是直线**，对角分布那种模式**再训练多久也分不开**——那是模型表达能力的限制。

★ 速查卡（考场上最常用的一串）

> 前向：$z = \Sigma w_i x_i + b$　　$p = \sigma(z) = 1/(1+e^{(-z)})$
> 损失：$y=1 \Rightarrow L = -\log p$　　$y=0 \Rightarrow L = -\log(1-p)$
> 信号：$\delta = \partial L/\partial z = p - y$　　← sigmoid + BCE 专属化简
> 梯度：$\partial L/\partial w_i = \delta \cdot x_i$　　$\partial L/\partial b = \delta$
> 更新：$\theta_new = \theta - \eta \cdot \partial L/\partial \theta$　← 全部梯度用原参数算完再一起更新